section, we'll explore the features that make Cedar unique and effective for building secure, real-time access control systems.

7.4.1 Strong data types

Cedar provides a variety of primitive types: booleans, strings, longs (64-bit integers), and composite types like sets and records—along with extensions such as datetime, decimal, duration, and ipaddr, allowing safe checks like `context.device.ip == ip("192.168.0.10")` or conditional logic based on timestamps.

Cedar is a statically validated, strongly typed policy language. All entities, actions, attributes, and relationships are defined in a schema, and policies are type-checked against that schema before they can be evaluated. Strong typing prevents misuse, such as comparing strings to numbers or mixing types in sets, catches errors at runtime, and reduces unpredictable behavior in production.

7.4.2 Rich entity model

Cedar supports first-class entity references, meaning that principals and resources aren't just strings or labels—they're structured objects. This enables natural, readable expressions like this:

```
resource.owner == principal
```

In this case, `principal` and `resource.owner` are entity references that Cedar can reason about, compare, and connect. Policies can express relationships (such as ownership or team membership) without needing to pass those links in every request. Instead, Cedar evaluates them against any entity data loaded at runtime.

This model mirrors how most real-world systems organize access: Employees own documents, managers oversee teams, and services act on behalf of users. Cedar's entity model allows you to directly represent and query those relationships, making policies more expressive, easier to maintain, and simpler to audit.